# BULK CERTIFICATE GENERATOR

Technical Project Documentation

A FastAPI-based REST API that accepts a bulk list of recipients and generates individual PDF certificates from a predefined template. The system tracks job progress, stores certificate status in a relational database, and allows generated certificates to be retrieved individually.

## PROJECT OVERVIEW

This document presents the implementation, API behavior, processing flow, testing, design decisions, and submission checklist for the project.

| | | | |
|---|---|---|---|
| **Core API** | FastAPI | **Database** | SQLite + SQLAlchemy |
| **Validation** | Pydantic | **PDF Generation** | ReportLab |
| **Processing** | BackgroundTasks | **Testing** | pytest |

## KEY CAPABILITIES

- Bulk certificate generation from a recipient list
- Input validation using Pydantic
- REST API built with FastAPI
- SQLite relational database using SQLAlchemy
- Background processing with job status and progress tracking
- Independent certificate failure handling
- Generated PDF retrieval and automated testing
- Interactive API documentation through Swagger UI

# 01 Technology & Project Structure

## Technology Stack

| Technology | Purpose |
|---|---|
| Python 3 | Application development language |
| FastAPI | REST API framework |
| SQLAlchemy | ORM / database access |
| SQLite | Relational database |
| Pydantic | Request validation |
| ReportLab | PDF certificate generation |
| pytest | Automated testing |
| Uvicorn | Development server |

## Project Structure

```
bulk-certificate-generator/
■■■ app/
■   ■■■ main.py
■   ■■■ database.py
■   ■■■ models.py
■   ■■■ schemas.py
■   ■■■ generator.py
■   ■■■ worker.py
■■■ generated/
■■■ test_api.py
■■■ README.md
■■■ requirements.txt
■■■ .gitignore
■■■ certificates.db
```

**Repository hygiene** **Do not commit:** venv/, generated/, __pycache__/, .pytest_cache/, and certificates.db.

## Setup

**1. Create and activate the virtual environment**

```
python -m venv venv
venv\Scripts\activate
```

**2. Install dependencies**

```
pip install -r requirements.txt
```

# 02 Running the Application & API

## Run the Application

From the project root:

```
uvicorn app.main:app --reload
```

| Service | Address |
|---|---|
| API | http://127.0.0.1:8000 |

| Service | Address |
| --- | --- |
| Swagger API documentation | http://127.0.0.1:8000/docs |

### Create a Bulk Certificate Job

**POST /jobs**

Example request:

```
{
  "event_name": "Python Workshop 2026",
  "certificate_date": "2026-10-08",
  "recipients": [
    {"name": "Madhuri", "email": "madhuri@example.com"},
    {"name": "Rahul", "email": "rahul@example.com"}
  ]
}
```

Example response:

```
{
  "job_id": 1,
  "status": "PENDING",
  "total": 2
}
```

**HTTP behavior** The endpoint returns **HTTP 202 Accepted** because certificate generation is processed as a background task.

# 03 API Endpoints & Validation

### Check Job Status

**GET /jobs/{job_id}**

```
{
  "job_id": 1,
  "status": "COMPLETED",
  "total": 2,
  "successful": 2,
  "failed": 0,
  "progress": "100%"
}
```

Possible job statuses:

- **PENDING**
- **PROCESSING**
- **COMPLETED**
- **COMPLETED_WITH_ERRORS**

### List Certificates for a Job

**GET /jobs/{job_id}/certificates**

```
[
  {
    "certificate_id": 1,
    "recipient_name": "Madhuri",
    "recipient_email": "madhuri@example.com",
    "status": "SUCCESS",
    "file_path": "generated/certificate_1.pdf",
    "error_message": null
  }
]
```

## Retrieve an Individual Certificate

**GET /certificates/{certificate_id}**

If the certificate was generated successfully, the API returns the generated PDF file.

## Input Validation

The API validates incoming recipient data before creating a job.

| Invalid input | Expected behavior |
|---|---|
| Invalid email address | Request rejected |
| Missing recipient name | Request rejected |
| Missing email address | Request rejected |
| Empty recipient list | Request rejected |

**Validation response** Invalid requests are rejected by FastAPI/Pydantic with **HTTP 422 Unprocessable Entity**.

# 04 Certificate Generation & Failure Handling

## Certificate Generation

Certificates are generated using ReportLab. The predefined certificate template contains:

- Certificate title
- Recipient name
- Event name
- Certificate date
- Certificate ID

Each generated certificate is saved as:

```
generated/certificate_<certificate_id>.pdf
```

## Bulk Processing and Failure Handling

Each certificate is processed independently. The worker wraps individual certificate generation in a try/except block.

If one certificate fails:

- The certificate is marked as **FAILED**.
- The error message is stored in the database.
- The job's failed counter is increased.
- Processing continues for the remaining recipients.

**Key behavior** A single bad certificate does not stop the entire bulk job.

## Example: 10 certificates requested and 1 fails

| Metric | Result |
|---|---|
| Total | 10 |
| Successful | 9 |
| Failed | 1 |
| Progress | 100% |
| Status | COMPLETED_WITH_ERRORS |

# 05 Database Design & Progress Tracking

## Database Design

The application uses two main database tables.

| Table | Fields |
| --- | --- |
| Job | id, status, total, successful, failed, created_at |
| Certificate | id, job_id, recipient_name, recipient_email, status, file_path, error_message |

## Relationship

```
Job
 |
 +---- Certificate
 |
 +---- Certificate
 |
 +---- Certificate

One job can contain many certificates.
```

## Progress Tracking

Progress is calculated using:

```
processed = successful + failed
progress = (processed / total) * 100
```

The database is updated after each certificate so that the current state of the job can be queried.

| | |
| --- | --- |
| **Tracking model** | Successful and failed certificate counts together determine the processed total and current progress. |

# 06 Background Processing & End-to-End Flow

## Background Processing

FastAPI BackgroundTasks is used to process certificate generation after the API accepts the request.

Simplified flow:

```
Client
  |
  | POST /jobs
  v
FastAPI
  |
  | Create Job + Certificates
  v
Database
  |
  | Start background task
  v
Worker
  |
  +--> Certificate 1 --> SUCCESS
  |
  +--> Certificate 2 --> SUCCESS
  |
  +--> Certificate 3 --> FAILED
  |
  v
Database updated
```

### End-to-End Flow

**01** Client sends a bulk recipient list to **POST /jobs**.

**02** FastAPI validates the request.

**03** A job is created in the database.

**04** Certificate records are created for each recipient.

**05** A background task starts processing the job.

**06** Each certificate is generated independently.

**07** Successful certificates are saved as PDF files.

**08** Failed certificates are marked as **FAILED** with an error message.

**09** Job progress is updated after each certificate.

**10** Client can check job status using **GET /jobs/{job_id}**.

**11** Client can list generated certificates using **GET /jobs/{job_id}/certificates**.

**12** Client can retrieve an individual PDF using **GET /certificates/{certificate_id}**.

| | |
|---|---|
| **Production note** | For a production system with a large number of certificates, a durable task queue such as **Celery/RQ with Redis** would be a better choice. |

# 07 Testing & Design Decisions

## Testing

The project includes automated tests covering the main assignment requirements:

- Job creation
- Input validation
- Certificate generation
- Job progress
- Individual certificate failure

Run the tests with:

```
pytest test_api.py -v
```

Expected result:

```
5 passed
```

If a direct certificate download/retrieval test has also been added, the total number of passing tests may be higher.

## Design Decisions

| Decision | Reason |
|---|---|
| FastAPI | Simple REST API development, automatic request validation, Swagger/OpenAPI documentation, background task support, and good developer experience. |
| SQLite | Relational database that is simple to set up and does not require a separate database server; suitable for the assignment. |
| SQLAlchemy | Provides a clean ORM layer and keeps database operations separate from API logic. |
| BackgroundTasks | Keeps the API responsive while certificate files are generated and is sufficient for the scope of the assignment. |

| | |
|---|---|
| **Production alternatives** | For production, **PostgreSQL** is a stronger database choice, and a dedicated task queue is preferred for high-volume workloads. |

# 08 Production Improvements & Submission Checklist

## Production Improvements

- PostgreSQL instead of SQLite
- Celery/RQ with Redis for durable background jobs
- Object storage such as S3 for generated certificates
- Authentication and authorization
- Rate limiting
- Structured logging
- Better HTTP error responses
- Job cancellation and retry support
- Database indexes for large datasets
- Docker/container deployment
- Monitoring and health checks
- More comprehensive integration tests

## Submission Checklist

Before submitting the project, make sure the repository contains:

```
app/
test_api.py
README.md
requirements.txt
.gitignore
```

Do not submit:

```
venv/
__pycache__/
.pytest_cache/
generated/
certificates.db
```

## Final Verification

```
pytest test_api.py -v
uvicorn app.main:app --reload
```

Then open:

```
http://127.0.0.1:8000/docs
```

and verify that the API is working.

## Conclusion

This project demonstrates a complete bulk certificate generation workflow using FastAPI, SQLAlchemy, SQLite, Pydantic, and ReportLab. It supports validation, background processing, progress tracking, independent failure handling, PDF generation, certificate retrieval, and automated testing.